

Інструкція: CI/CD для Flutter-проєкту

Покроковий гайд для налаштування автоматичного деплою **Android** → **Google Play Internal** та **iOS** → **TestFlight** після merge в main.

Референс-проєкт: fastlane_demo_ci

Що отримаєте

Подія	Результат
Pull Request → main	CI: тести + перевірка збірки
Merge / push в main	Auto-deploy: Google Play Internal + TestFlight
Ручний workflow	Production у Play Store / App Store

Секрети не комітяться в git — лише в GitHub Secrets.

Передумови

- Flutter SDK
 - Ruby 3.x + Bundler (gem install bundler)
 - Mac (для одноразового налаштування iOS match)
 - Акаунти: Google Play Console, Apple Developer, App Store Connect
 - GitHub repo з увімкненими Actions
-

Крок 1. Взяти шаблон

Варіант А — скопіювати репозиторій

```
git clone git@github.com:ErikaDyendyeshy/fastlane_demo_ci.git my-app
cd my-app
rm -rf .git
git init
git remote add origin git@github.com:<org>/<your-repo>.git
```

Варіант В — додати CI/CD у існуючий Flutter-проєкт

Скопіюйте з шаблону:

```
.github/workflows/          → ci.yml, deploy.yml, cd-android.yml, cd-ios.yml
.github/actions/            → setup-android-signing, setup-app-store-connect
Gemfile + Gemfile.lock
android/fastlane/
ios/fastlane/
```

Потім у корені проєкту:

```
bundle install
```

Крок 2. Налаштувати Bundle ID / applicationId

Усі місця мають збігатися — інакше деплой впаде.

Замініть `com.fastlane.ci` на свій ID (наприклад `com.company.myapp`).

Файл	Що змінити
<code>android/app/build.gradle.kts</code>	<code>namespace, applicationId</code>
<code>android/fastlane/Appfile</code>	<code>package_name(...)</code>
<code>ios/Runner.xcodeproj/project.pbxproj</code>	<code>PRODUCT_BUNDLE_IDENTIFIER</code>
<code>ios/fastlane/Appfile</code>	<code>app_identifier(...)</code>
<code>ios/fastlane/Matchfile</code>	<code>app_identifier([...])</code>
<code>ios/fastlane/Fastfile</code>	КОНСТАНТА <code>APP_IDENTIFIER</code>
<code>android/.../MainActivity.kt</code>	<code>package</code> + шлях до файлу

Також створіть додатки в консолях з **тим самим ID**:

- Google Play Console → Create app
- App Store Connect → Apps → New App
- Apple Developer → Identifiers → App ID

Крок 3. GitHub Environment

1. GitHub → repo → **Settings** → **Environments**
2. **New environment** → назва: `production`
3. (Опційно) додайте protection rules — хто може деплоїти

Крок 4. Android

4.1. Upload keystore (один раз)

```
keytool -genkey -v \  
-keystore upload-keystore.jks \  
-keyalg RSA -keysize 2048 -validity 10000 \  
-alias upload
```

Пароль ≥ 6 символів. **Збережіть keystore і паролі назавжди** — без них не оновите додаток у Play Store.

4.2. Google Play Service Account

1. Google Cloud Console → **IAM** → **Service Accounts** → Create
2. Створіть JSON ключ → завантажте `google-play-key.json`
3. Play Console → **Setup** → **API access**
4. Прив'яжіть Google Cloud проєкт
5. Надайте service account права **Release manager**

4.3. GitHub Secrets (Android)

Settings → Secrets and variables → Actions → **New repository secret:**

Secret	Значення
ANDROID_KEYSTORE_BASE64	<code>base64 -i upload-keystore.jks \ pbcopy</code>
ANDROID_KEYSTORE_PASSWORD	пароль keystore
ANDROID_KEY_ALIAS	upload
ANDROID_KEY_PASSWORD	пароль ключа
GOOGLE_PLAY_JSON_KEY_BASE64	<code>base64 -i google-play-key.json \ pbcopy</code>

Не комітьте `upload-keystore.jks` і `google-play-key.json` у `git`.

Крок 5. iOS

5.1. App Store Connect API Key

- App Store Connect → **Users and Access** → **Integrations** → **App Store Connect API**
- Generate API Key** (Admin або App Manager)
- Завантажте `AuthKey_XXXXX.p8`
- Запишіть **Key ID** та **Issuer ID** (на тій же сторінці)

5.2. Match — сертифікати (один раз, на Mac)

- Створіть **приватний** GitHub репо для сертифікатів, напр. `my-org/my-app-match`
- Оновіть `ios/fastlane/Matchfile`:

```
git_url("git@github.com:<org>/<my-app-match>.git")
username("your-apple-id@email.com")
app_identifier(["com.company.myapp"])
```

- Запустіть `match` (`Matchfile` вже є — `match init` не потрібен):

```
cd ios
bundle exec fastlane match appstore
```

Запитає: - **Passphrase for Match storage** — придумайте і запишіть → `MATCH_PASSWORD` - **Apple ID password** — або app-specific password (якщо 2FA) - **Mac login password** — пароль від входу в macOS (для Keychain)

Після успіху сертифікати з'являться у `match` репо.

5.3. SSH ключ для CI

```
ssh-keygen -t ed25519 -C "github-actions-match" -f ~/.ssh/match_deploy_key -N ""
cat ~/.ssh/match_deploy_key.pub # → Deploy key у match repo
cat ~/.ssh/match_deploy_key | pbcopy # → Secret MATCH_SSH_PRIVATE_KEY
```

GitHub → match repo → **Settings** → **Deploy keys** → Add deploy key (read-only достатньо для CI).

5.4. GitHub Secrets (iOS)

Secret	Значення
APP_STORE_CONNECT_API_KEY_BASE64	base64 -i AuthKey_XXXXX.p8 \ pbcopy
APP_STORE_CONNECT_KEY_ID	Key ID (напр. 2Q4L88F8S6)
APP_STORE_CONNECT_ISSUER_ID	Issuer ID
APPLE_ID	Apple ID email
APPLE_TEAM_ID	Team ID з developer.apple.com/account → Membership
MATCH_GIT_URL	git@github.com:<org>/<my-app-match>.git
MATCH_PASSWORD	passphrase з match
MATCH_SSH_PRIVATE_KEY	приватний SSH ключ (повний вміст .pem)

Крок 6. Чеклист перед першим деплоєм

- [] Bundle ID однаковий скрізь (Android + iOS + консолі)
- [] GitHub Environment "production" створено
- [] 5 Android secrets додано
- [] 8 iOS secrets додано
- [] match appstore пройшов успішно локально
- [] Deploy key додано в match repo
- [] Код запущено в main

Крок 7. Перший деплой

```
git add .
git commit -m "Setup CI/CD"
git push -u origin main
```

GitHub → **Actions** → workflow **Deploy**:

- Deploy Android (Internal) → Google Play Internal testing
- Deploy iOS (TestFlight) → TestFlight

Нічого натискати не потрібно — стартує автоматично після push в main.

Щоденна робота

Новий білд (beta)

1. Збільште build number у pubspec.yaml:

```
version: 1.0.1+10 # +10 — обов'язково більший за попередній
```

2. Commit + merge в main:

```
git commit -am "Release 1.0.1+10"  
git push origin main
```

3. Перевірте Actions → Deploy

Production (коли beta протестовано)

Платформа	Дія
Android	Actions → CD Android (Production) → Run workflow
iOS	Actions → CD iOS (Production) → Run workflow

Локальна розробка

```
flutter pub get  
bundle install
```

Android (локальний підпис):

```
cp android/key.properties.example android/key.properties  
# відредагуйте паролі, покладіть keystore в android/app/upload-keystore.jks
```

```
cd android  
bundle exec fastlane build # AAB
```

iOS:

```
cd ios  
bundle exec fastlane build_ci # без підпису  
bundle exec fastlane beta # TestFlight (потрібен match + API key  
локально)
```

Типові помилки

Помилка	Причина	Рішення
Keystore file ... app/ app/... not found	неправильний storeFile	storeFile=upload- keystore.jks (не app/...)
Package not found / does not exist	bundle ID не збігається	вирівняйте ID у проєкті та консолі
google-play-key.json not found	неправильний шлях	fastlane/google-play- key.json
App Development provisioning profiles	Flutter/gym шукає dev profile	використовуйте Fastfile з шаблону (gym + match)
PROVISIONING_PROFILE_SPECIFIER = match	робили в імені profile без лапок	не передавайте profile через xcargs string
Missing password for user	TestFlight без API key	api_key: у upload_to_testflight
Duplicate build	той самий build number	збільште +N у pubspec.yaml

Структура CI/CD

```
.github/
├── workflows/
│   ├── ci.yml           # PR: analyze, test, build verify
│   ├── deploy.yml       # main: auto-deploy Internal + TestFlight
│   ├── cd-android.yml   # ручний Production (Android)
│   └── cd-ios.yml        # ручний Production (iOS)
├── actions/
│   ├── setup-android-signing/
│   └── setup-app-store-connect/
├── android/fastlane/     # lanes: build, beta, release
└── ios/fastlane/         # lanes: build, beta, release + Matchfile
```

Корисні посилання

- Fastlane
- Flutter deployment
- match (code signing)
- Google Play AAB